Compulsory In-class Assignment (Consul-based Localization)

SEP2 (Week 1)/AD

🛒 Java Shopping Cart with Localization

Goal:

Build a simple Java console app that calculates the total cost of items in a shopping cart with localization and UTF-8 encoding.

What to do:

1. Ask the user for item prices and quantities, then calculate the total cost.
2. Add localization for:
   - English
   - Finnish
   - Swedish
   - Japanese
3. Create a Docker image for your app and make sure it runs correctly.
4. Set up automated tests and a CI/CD pipeline (e.g., Jenkins) to:
   - Build and test the app
   - Create and push the Docker image to Docker Hub
5. Finally, test your Docker image using:
   👉 https://labs.play-with-docker.com/

Focus points:

- Use UTF-8 encoding
- Implement basic localization
- Show CI/CD and Docker skills

---

1. Java Console Application Development

Objective: Create a Java console application that performs the following tasks:

1. Prompt the user to enter the number of items they want to purchase.
2. For each item, ask the user for the price and quantity.
3. Calculate the total cost of each item (price × quantity).
4. Calculate the total cost of all items in the shopping cart.
5. Display the total cost of the shopping cart to the user.
6. Implement localization so that the program can display messages in Finnish, Swedish, and English based on the user's language selection.

Inputs:

- Item prices (per item).
- Item quantities (per item).

Assumption: The program should assume there is no discount applied, but you may add features like discount or tax calculation if desired.

---

2. Localization

Objective: Implement localization support for Finnish, Swedish, and English in your Java application.

Requirements:

- The program should ask the user to select their preferred language when starting the application (English, Finnish, or Swedish, Japanses).
- Based on the selection, display all messages and prompts in the corresponding language.
- You need to create three properties files for localization:
  - MessagesBundle_en_US.properties (for English) as a default language
  - MessagesBundle_sv_SE.properties (for Swedish)
  - MessagesBundle_fi_FI.properties (for Finnish)
  - MessagesBundle_ja_JP.properties (For Japanese)

For example:

- English Messages:
  - "Enter the number of items to purchase:"
  - "Enter the price for item:"
  - "Enter the quantity for item:"
  - "Total cost:"
- Finnish Messages:
  - "Syötä ostettavien tuotteiden määrä:"
  - "Syötä tuotteen hinta:"
  - "Syötä tuotteen määrä:"
  - "Kokonaishinta:"
- Swedish Messages:
  - "Ange antalet varor att köpa:"
  - "Ange priset för varan:"
  - "Ange mängden varor:"
  - "Total kostnad:"
- Japanese Message:
  - 「購入する商品の数を入力してください:」
  - 「商品の価格を入力してください:」
  - 「商品の数量を入力してください:」
  - 「合計金額:」

---

3. Write Unit Tests and Coverage Tests

Objective: Create unit tests using JUnit 5 to verify the key functionalities of the application (e.g., calculating the total cost for each item, calculating the total cost of the cart).

Requirements:

- Write test cases to verify that each calculation (individual item cost and total cart cost) is correct.
- Apply JaCoCo (Java Code Coverage Tool) to measure test coverage and ensure that the core logic is properly tested.

---

4. Set Up GitHub and Push the Code

Objective: Initialize a Git repository for your project and push your code to GitHub.

Requirements:

- Initialize a Git repository in your project folder.
- Commit your changes and push the project to a GitHub repository.
- Organize your project structure and code clearly in the repository.

---

5. Implement Jenkins Pipeline with Jenkinsfile

Objective: Set up an automated build and test pipeline using Jenkins.

Requirements:

- Create a Jenkinsfile to define the pipeline for building, testing, and deploying your application.
- Configure the Jenkins pipeline to run automated tests (unit tests created in step 2) using the Jenkinsfile.
- Push the Jenkinsfile to your GitHub repository.
- Configure Jenkins to run the pipeline from the SCM (GitHub repository).

---

6. Containerize the Application with Docker

Objective: Containerize your Java application using Docker.

Requirements:

- Write a Dockerfile to define the environment for running the Java application.
- Build the Docker image for your Java application.
- Run the Docker image locally to verify that your application works as expected inside the container.

---

7. Update Jenkinsfile to Deploy the Image to Docker Hub

Objective: Modify your Jenkinsfile to add an additional stage that deploys the Docker image to Docker Hub.

Requirements:

- Ensure your Jenkinsfile contains a stage that builds and deploys the Docker image to Docker Hub.
- Set up a Docker Hub account and push your Docker image to your repository.

---

8. Moodle Submission Requirements

1. ***GitHub Repo Link****:* Submit the URL to your GitHub repository containing all the project files.
2. ***Screenshot of Lab Demo Execution***: Submit a screenshot (preferably as a PDF) demonstrating the successful execution of your image. Make sure your name has appeared in the screenshot, example (Amir Dirin). If your Docker Hub repository includes your account name (e.g., amirdirin/myimagerepo) in the screenshot, you **do not need** to add your name separately.

Evaluation Criteria:

- Functionality: The application works as expected, correctly calculating the total cost based on price and quantity.
- Localization: The program correctly switches between Finnish, Swedish, and English based on user input.
- Unit Tests: Unit tests are implemented, covering all the key functionality with JaCoCo coverage.
- GitHub and Version Control: The project is pushed to GitHub and follows standard Git practices.
- Jenkins Pipeline: The pipeline is set up, including automated tests.
- Dockerization: The application is successfully containerized, and the Docker image is pushed to Docker Hub.